



VOY & VUELVO

DOCUMENTACIÓN TÉCNICA DE ARQUITECTURA DE MICROSERVICIOS

*Plataforma para la gestión de rutas de
trekking basada en microservicios*



Evaluación Parcial N°1 — JVV0101

Java: Diseño y Construcción de
Soluciones nativas en Nube



Integrantes:

Michel Sanhueza · Franco Araya



2026



Índice

1. Introducción
2. Requisitos funcionales y no funcionales
3. Justificación del enfoque de microservicios
4. Arquitectura general del sistema
5. Delimitación de los microservicios (responsabilidad única)
6. Comunicación entre microservicios e interdependencias
7. Patrones de diseño integrados
8. Seguridad
9. Flujos de comunicación y tecnologías cloud
10. Flujos de datos críticos
11. División lógica de los servicios
12. Conclusión

1. Introducción

1.1 Problemática

Las empresas de turismo aventura suelen administrar reservas, rutas y equipamiento mediante registros manuales o sistemas separados. Esto dificulta el control de cupos, la gestión de usuarios y el seguimiento de las reservas, generando errores en la disponibilidad de rutas, duplicidad de información y dificultades para coordinar los procesos de una actividad de trekking.

1.2 Propuesta

Voy & Vuelvo es una plataforma orientada a la gestión de rutas de trekking que centraliza la administración de usuarios, rutas, equipamiento, reservas y pagos en una única solución, construida sobre una arquitectura de microservicios. Esto permite mejor control de reservas, gestión centralizada, mayor escalabilidad y comunicación entre servicios especializados.

2. Requisitos funcionales y no funcionales

Los requisitos se identificaron a partir de los cinco dominios de negocio del sistema: usuarios, rutas, equipamiento, reservas y pagos. Se clasifican a continuación según su tipo y se priorizan según su impacto directo en el diseño de la arquitectura distribuida.

2.1 Requisitos funcionales (RF)

ID	Requisito	Microservicio asociado	Prioridad
RF-01	Registrar, consultar y eliminar usuarios (nombre, email único, teléfono).	ms-usuario	Alta
RF-02	Administrar el catálogo de rutas de trekking (ubicación, dificultad, precio, cupo máximo).	ms-ruta	Alta
RF-03	Gestionar equipamiento disponible para arriendo, filtrable por dificultad y disponibilidad.	ms-equipamiento	Media
RF-04	Crear reservas validando usuario, cupos de la ruta y cantidad de personas.	ms-reserva	Alta
RF-05	Descontar/reponer cupos de una ruta automáticamente al reservar o cancelar.	ms-ruta / ms-reserva	Alta
RF-06	Generar y confirmar el pago asociado a una reserva (PENDIENTE, PAGADO, GARANTÍA_RETENIDA).	ms-pago	Alta
RF-07	Recomendar equipamiento según la ruta seleccionada.	ms-equipamiento → ms-ruta	Media
RF-08	Autenticar usuarios mediante login y emitir un token de acceso.	api-getaway	Alta

2.2 Requisitos no funcionales (RNF)

ID	Requisito	Criterio	Cómo se aborda en el diseño
RNF-01	El sistema debe permitir escalar cada dominio de forma independiente.	Escalabilidad	Cada microservicio se despliega en un contenedor Docker propio y puede replicarse sin afectar a los demás.
RNF-02	El sistema debe mantenerse operativo aunque un servicio individual falle o se actualice.	Disponibilidad	Registro dinámico en Eureka; el Gateway enruta solo a instancias registradas y activas.
RNF-03	El código debe poder mantenerse y extenderse sin impactar otros dominios.	Mantenibilidad	Cada microservicio tiene su propio código fuente, base de datos lógica y responsabilidad única.
RNF-04	El acceso a la información debe ser centralizado y trazable.	Seguridad	Autenticación JWT en el API Gateway; validación de datos de entrada con Bean Validation en cada servicio.
RNF-05	La documentación de la API debe estar disponible para pruebas e integración.	Usabilidad técnica	Swagger/OpenAPI expuesto en cada microservicio y agregado en el Gateway.

3. Justificación del enfoque de microservicios

Se evaluó un enfoque monolítico frente a uno de microservicios para dar respuesta a los requisitos anteriores:

Criterio	Monolito	Microservicios (elegido)
Escalabilidad	Se escala la aplicación completa, aunque solo un módulo tenga alta demanda (p. ej. reservas en temporada alta).	Cada servicio (ms-reserva, ms-ruta, etc.) escala de forma independiente según su carga.
Despliegue	Un cambio en cualquier módulo obliga a redespigar todo el sistema.	Cada microservicio se construye, prueba y despliega de forma aislada (Docker por servicio).
Mantenibilidad	El código de usuarios, rutas, pagos, etc. queda acoplado en una sola base de código.	Responsabilidad única por dominio de negocio; los equipos pueden trabajar en paralelo.
Tolerancia a fallos	Un error no controlado puede afectar a toda la aplicación.	Un fallo en un microservicio (p. ej. ms-pago) no impide el funcionamiento de los demás.

Dado que Voy & Vuelvo tiene dominios de negocio claramente diferenciados (usuarios, rutas, equipamiento, reservas, pagos) con distintos patrones de uso y crecimiento esperado, la arquitectura de microservicios responde mejor a los

requisitos de escalabilidad, disponibilidad y mantenibilidad identificados en la sección 2.

4. Arquitectura general del sistema

El sistema está compuesto por siete componentes desplegados como contenedores Docker independientes, orquestados mediante docker-compose. El diagrama de arquitectura completo se entrega en el archivo adjunto arquitectura.drawio (editable en draw.io / diagrams.net).

Componente	Puerto	Rol
Cliente / Frontend	—	Consume la API a través del API Gateway.
api-getaway	8080	Punto de entrada único. Autenticación JWT, enrutamiento y agregación de documentación Swagger.
eureka-server	8761	Registro y descubrimiento de servicios (Service Registry).
ms-usuario	8084	Gestión de usuarios.
ms-equipamiento	8081	Gestión de equipamiento de arriendo.
ms-ruta	8086	Gestión de rutas de trekking y control de cupos.
ms-reserva	8083	Orquestación de reservas: valida usuario, descuenta cupos y genera el pago.
ms-pago	8085	Gestión de pagos asociados a una reserva.
MySQL	3306	Motor de base de datos, con una base de datos lógica por microservicio (bd_usuario, bd_equipamiento, bd_ruta, bd_reserva, bd_pago, bd_gateway).

5. Delimitación de los microservicios (responsabilidad única)

Cada microservicio se delimitó según su dominio funcional en el negocio, evitando que dos servicios compartan la misma responsabilidad:

Microservicio	Entidad principal	Responsabilidad única
ms-usuario	Usuario (nombre, email, teléfono)	Es la única fuente de verdad sobre la identidad y datos de contacto de los usuarios.
ms-equipamiento	Equipamiento (dificultad, capacidad, valor de arriendo)	Administra exclusivamente el catálogo e inventario de equipamiento disponible para arriendo.
ms-ruta	Ruta (ubicación, dificultad, precio, cupoMaximo)	Es dueño del catálogo de rutas y del control de cupos disponibles por ruta.
ms-reserva	Reserva (usuariold, rutald, cantidadPersonas, estadoPago)	Orquesta el proceso de reserva: valida al usuario, gestiona cupos y solicita el pago, sin duplicar datos de otros dominios.
ms-pago	Pago (idReserva, monto, estado)	Es la única fuente de verdad sobre el estado de pago de una reserva.

Ningún microservicio persiste datos que pertenezcan a otro dominio: ms-reserva, por ejemplo, no guarda los datos del usuario ni de la ruta, solo sus identificadores (usuariold, rutald), y obtiene el resto de la información mediante comunicación con el microservicio dueño de esos datos.

6. Comunicación entre microservicios e interdependencias

La comunicación síncrona entre microservicios se implementa con Feign Client. Esto mantiene bajo acoplamiento porque cada servicio solo conoce la interfaz (DTO + endpoint) del servicio que consume, no su implementación interna ni su base de datos.

Origen	Destino	Comunicación (Feign)	Propósito
ms-equipamiento	ms-ruta	GET /api/rutas/{id}, GET /api/rutas	Obtener datos de la ruta para recomendar equipamiento (RF-07).
ms-ruta	ms-equipamiento	GET /api/equipamiento/interno/**	Listar equipamiento disponible asociado a una ruta.
ms-reserva	ms-usuario	GET /api/usuarios/{id}	Validar que el usuario exista antes de crear la reserva.
ms-reserva	ms-ruta	PUT /descontar-cupos, /aumentar-cupos	Descontar cupos al reservar y reponerlos al cancelar/eliminar.
ms-reserva	ms-pago	POST /api/pagos	Generar automáticamente el pago (estado PENDIENTE) al confirmar la reserva.

ms-reserva actúa como orquestador del flujo de negocio, pero la reutilización de componentes se mantiene alta: ms-ruta y ms-equipamiento son consumidos por más de un servicio sin necesidad de duplicar su lógica, y cada Feign Client se comunica únicamente a través de DTOs desacoplados de las entidades JPA internas.

7. Patrones de diseño integrados

Patrón	Dónde se aplica	Aporte a la arquitectura
API Gateway	api-getaway (puerto 8080)	Punto único de entrada; centraliza autenticación, enrutamiento por prefijo de path y agregación de documentación Swagger de los 5 microservicios. Mejora la seguridad y la mantenibilidad frente al cliente.
Service Registry / Discovery	eureka-server (puerto 8761)	Cada microservicio y el Gateway se registran dinámicamente. Permite localizar instancias sin direcciones fijas y sostiene la disponibilidad ante reinicios o escalado.
Client-Side Communication (Feign Client)	ms-equipamiento, ms-ruta, ms-reserva, ms-pago	Encapsula las llamadas HTTP entre servicios en interfaces declarativas tipadas, reduciendo el acoplamiento respecto a llamadas REST manuales.

Patrón	Dónde se aplica	Aporte a la arquitectura
Database per Service	Todos los microservicios (bd_usuario, bd_ruta, bd_equipamiento, bd_reserva, bd_pago)	Cada servicio posee su propio esquema de base de datos, evitando acoplamiento a nivel de datos entre dominios.

8. Seguridad

Punto de seguridad	Ubicación	Detalle
Autenticación	api-getaway — /auth/login, /auth/register	AuthService valida credenciales y JwtUtil emite un token firmado con clave y expiración configuradas (24 h).
Autorización / filtro de peticiones	api-getaway — JwtFilter	Filtro WebFlux que intercepta las peticiones entrantes en el Gateway antes de enrutarlas a los microservicios.
Validación de datos de entrada	Los 5 microservicios (Bean Validation / Jakarta)	Anotaciones @NotBlank, @NotNull, @Email, @Positive, @Min en las entidades evitan datos inválidos o incompletos (p. ej. email inválido, cantidadPersonas ≤ 0).
Manejo centralizado de errores	GlobalExceptionHandler en cada microservicio	Traduce excepciones de negocio (p. ej. CuposInsuficientesException, RecursoNoEncontradoException) en respuestas HTTP controladas, evitando exponer detalles internos.

9. Flujos de comunicación y tecnologías cloud

Tecnología	Uso en el proyecto
Spring Boot	Framework base de los 7 servicios (microservicios, Gateway y Eureka).
Spring Cloud Gateway	Enrutamiento reactivo en api-getaway hacia cada microservicio.
Spring Cloud Netflix Eureka	Registro y descubrimiento de servicios.
OpenFeign	Comunicación síncrona declarativa entre microservicios.
MySQL 8	Persistencia de datos, una base lógica por microservicio, contenedor compartido.
Docker / docker-compose	Empaquetado y orquestación de los 7 contenedores del sistema.

Tecnología	Uso en el proyecto
Swagger / springdoc-openapi	Documentación interactiva de cada API, agregada en el Gateway.
JUnit + Mockito	Pruebas unitarias de servicios y controladores.

Integración externa: el único punto de entrada expuesto al exterior es el API Gateway (puerto 8080). Los microservicios, Eureka y MySQL solo son accesibles dentro de la red interna de Docker, lo que reduce la superficie de ataque del sistema.

10. Flujos de datos críticos

El flujo más crítico del sistema es la creación de una reserva, porque orquesta a cuatro servicios en una sola operación:

1. El cliente envía la solicitud al API Gateway (POST /api/reservas).
2. El Gateway enruta la petición a ms-reserva.
3. ms-reserva valida al usuario mediante ms-usuario (Feign).
4. ms-reserva descuenta los cupos de la ruta mediante ms-ruta (Feign); si no hay cupos suficientes, lanza CuposInsuficientesException.
5. ms-reserva genera el pago (estado PENDIENTE) mediante ms-pago (Feign).
6. La reserva queda registrada en bd_reserva y la respuesta vuelve al cliente a través del Gateway.

Puntos de atención identificados:

- Cuello de botella / punto crítico: ms-reserva, por ser el orquestador que depende de la disponibilidad de tres servicios (ms-usuario, ms-ruta, ms-pago) para completar una sola operación.
- Punto de monitoreo: la base de datos MySQL compartida por todos los servicios (mismo contenedor, distintos esquemas), recomendable de monitorear en throughput y conexiones concurrentes.
- Consistencia: si el descuento de cupos en ms-ruta tiene éxito pero la llamada a ms-pago falla, la reserva queda creada sin pago generado; se recomienda incorporar reintentos o un mecanismo de compensación como mejora futura.

11. División lógica de los servicios

La división en 5 microservicios de negocio (usuario, equipamiento, ruta, reserva, pago) más Gateway y Eureka responde a los tres criterios definidos en los requisitos no funcionales:

- Escalabilidad: ms-reserva y ms-pago —los de mayor carga transaccional en temporada alta— pueden escalarse de forma independiente sin replicar ms-usuario o ms-equipamiento.
- Seguridad: al concentrar la autenticación en un único punto (api-getaway), se evita duplicar lógica de seguridad en cada microservicio; cada servicio, además, valida sus propios datos de entrada.

- **Mantenibilidad:** un cambio en las reglas de negocio de pagos (ms-pago) no requiere modificar, recompilar ni redespargar ms-usuario, ms-ruta o ms-equipamiento.

Mejoras recomendadas para una futura iteración: incorporar un patrón Circuit Breaker (p. ej. Resilience4j) en las llamadas Feign entre ms-reserva y sus dependencias, y restringir las rutas /api/** del Gateway para exigir JWT en producción.

12. Conclusión

Voy & Vuelvo implementa una arquitectura de microservicios funcional para la gestión de rutas de trekking, con cinco servicios de negocio delimitados por responsabilidad única, comunicación desacoplada mediante Feign Client, descubrimiento de servicios con Eureka y un punto de entrada centralizado y autenticado a través de API Gateway. El diseño responde a los requisitos de escalabilidad, disponibilidad y mantenibilidad identificados, dejando además documentadas las mejoras recomendadas (Circuit Breaker y endurecimiento de la seguridad en el Gateway) para una futura iteración del proyecto.